

From a paper algorithm to Python

Project: Python readiness portfolio

1. Prepare the purpose, prerequisites and materials

Teach the learning cycle

Primary target: Translate a short paper calculation into a Python script

Session · Sequence and minutes · Resources

Session: Session 1 · Understand · 45 min

Sequence and minutes: 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.

Resources: Sections 1–6; record the brief recall in the workbook.

Session: Session 2 · Practise · 45 min

Sequence and minutes: 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

Resources: Sections 7–10; use the matching workbook W2 and misconception check.

Session: Session 3 · Investigate and transfer · 45 min

Sequence and minutes: 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

Resources: Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building. Repeated copies of a case share one recorded attempt; they are not new evidence.

Case exposure check: this source example appears in ai-2/1, ai-3/23, python-bridge/1. Reuse a familiar

case for recall or modelling; verify that the case used to assess transfer has not already received feedback.

Retrieval answer guidance

Four packs contain six counters each. What steps find and show the total?

Record four and six, multiply them to get 24, then display 24. The order must make the required values available before use.

This is a first encounter with saved Python scripts, not a compressed implementation of the Grade 6 adviser. Teach assignment, multiplication and print only as needed; values and types follow in Week 2. Use a fresh run to avoid hidden interactive state. Reference: Python tutorial, An Informal Introduction to Python, <https://docs.python.org/3/tutorial/introduction.html> (assignment, arithmetic and print).

Outcomes

Translate a short paper calculation into a Python script

Predict and explain the order of executed statements

Keep expected and observed output separate when checking a program

Preparation

Teacher-prepared Python 3 editor with a new saved .py file and a visible output area; test Run before class

Paper counters or equal-group sketches and a blank expected/observed record

The printed script and workbook; no logins or personal data required

Prerequisite answer

Record four and six, multiply them to get 24, then display 24. The order must make the required values available before use.

End goal for the pictorial case

Trace from a paper algorithm to python and explain the changed program before running it.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

2. Explain and model the mechanism

In Grade 6 you traced rules on paper. Now you will write a small Python program and check what it actually does. Begin with a counter-packing calculation, not the whole activity adviser. The requirement is to find the total counters in equal packs and display that total. For this first program, the supplied quantities are whole, nonnegative numbers; it does not yet validate typed input.

Use a teacher-prepared Python 3 editor that can run a saved .py file. Open a new file called counters.py,

type the code exactly as shown in the next section, save it, and use the editor's Run command. Read the output area separately from the source editor. No account, AI tool, package installation or personal data is needed. If a device is unavailable, trace on paper and label the result predicted rather than executed.

Read the script from top to bottom. The equals sign assigns the value on its right to the name on its left. The star means multiplication. `print(total)` displays the value of `total`; it does not display the letters in the name. Type ordinary straight characters and keep the parentheses. The Python prompt symbols `>>>` are not part of this saved script.

Term · Meaning

Term: Algorithm

Meaning: An ordered procedure for solving a stated problem

Term: Program

Meaning: Instructions written in a language a computer can execute

Term: Interpreter

Meaning: Software that executes Python instructions

Term: Assignment

Meaning: Binding a name to the value of an expression

Term: Output

Meaning: Information a program displays or returns

A calculation from paper to output

Paper step · Python instruction · Effect

Paper step: Record pack count

Python instruction: `packs = 4`

Effect: Name holds 4

Paper step: Multiply two quantities

Python instruction: `total = packs * per_pack`

Effect: Name holds product

Paper step: Show the answer

Python instruction: `print(total)`

Effect: Displays the value

Assign `per_pack` before the multiplication. This diagram summarises the steps; the complete script below includes that assignment.

The third line assigns `total` but does not call `print`, so nothing is displayed. A first-line `print(total)` raises `NameError` in a fresh run because `total` has not been assigned. Exit: 11 is wrong for five groups of six; addition is a likely cause, and multiplication should give 30. Retain the old result and check the repaired file with both pack counts. Record whether the work was executed or traced.

Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a prediction, then explain the deciding step. These teacher notes contain solutions; do not reveal the independent case answer during modelling.

packs = 4 · packs is 4 · Nothing yet

packs = 4: `per_pack = 6`

packs is 4: `packs` is 4; `per_pack` is 6

Nothing yet: Nothing yet

packs = 4: `total = packs * per_pack`

packs is 4: `total` is 24

Nothing yet: Nothing yet

packs = 4: `print(total)`

packs is 4: Values are unchanged

Nothing yet: 24

Write 24 as the expected output before running. Then copy the actual output into a separate column. A matching result supports this case, not every possible input. A saved file may contain an old edit: save again and rerun the whole script. Do not type the expected number into the output record without running.

Why is there no displayed output after the third line? What changes if `print(total)` is placed first in a fresh run, before any assignment? Explain using which names exist.

Thinking move · Teacher prompt

Thinking move: Plan

Teacher prompt: What is given? What is the question? Which procedure fits these facts?

Thinking move: Do

Teacher prompt: Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.

Thinking move: Check

Teacher prompt: Does the output answer the question? Which assumption or limitation prevents a larger claim?

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

A pictorial worked case: From a paper algorithm to Python

Convert a seed-count algorithm. The paper plan counts seeds in identical trays, then adds a labelled spare tray.

The facts we will use

Python 3: `trays=4; per_tray=6; spare=2; total=trays*per_tray+spare; print(total)`. All quantities count seeds.

From a paper algorithm to Python

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Name the inputs

Four trays hold six seeds each; two extra seeds are separate.



2. Translate the plan

Multiply tray count by seeds per tray, then add the spare count.



3. Trace the assignment

$4 \times 6 + 2$ is 26; total stores 26 and print displays it.

CHANGE ONE THING

Change spare to 5 while keeping both tray inputs fixed.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Name the inputs

What happens and why: Four trays hold six seeds each; two extra seeds are separate.

Step: 2. Translate the plan

What happens and why: Multiply tray count by seeds per tray, then add the spare count.

Step: 3. Trace the assignment

What happens and why: $4 \times 6 + 2$ is 26; total stores 26 and print displays it.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Python follows the arithmetic instructions. This translation automates a declared counting rule; it does not train a model from examples.

What this example does not establish

The calculation assumes identical tray sizes and measured counts; it cannot detect a miscounted tray.

Change one thing in the pictured case

Change: Change spare to 5 while keeping both tray inputs fixed.

Prediction: The printed total becomes 29.

Reason: 4×6 remains 24; adding five gives 29.

Ask while showing the picture

Cover the result. Ask learners to name the inputs and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its evidence. Then change the supplied condition and compare. The picture is a representation; it does not document execution of a real AI model.

3. Demonstrate and guide practice

Time or stage · Teacher action

Time or stage: Session 1 start to 7 minutes

Teacher action: Retrieve equal-group multiplication and collect the prerequisite explanation.

Time or stage: 7 to 20 minutes

Teacher action: Model source versus output, save counters.py and translate each paper step with the diagram.

Time or stage: 20 to 33 minutes

Teacher action: Complete W1 and predict the trace before running the four-line program.

Time or stage: 33 to 45 minutes

Teacher action: Start W2 and distinguish observation from the expected answer; record device access.

Time or stage: Session 2 start to 8 minutes

Teacher action: Rerun with five packs and compare the two records.

Time or stage: 8 to 20 minutes

Teacher action: Use W3 and the guided question to diagnose a first-line print in a fresh run.

Time or stage: 20 to 35 minutes

Teacher action: Collect W4 independently before hints; observe whether each pupil can explain zero trays.

Time or stage: 35 to 45 minutes

Teacher action: Complete W5 and the exit. Save source and records; identify pupils needing editor support.

Use the worked example in Sections 3 and 6. Collect a prediction before revealing its result; ask the learner to point to each intermediate fact.

Offer four separate line cards, a source/output label and a multiplication sketch. Allow a paper trace when devices are unavailable, with execution practice still pending. Do not supply the independent transfer numbers' answers. Extension: explain why a plus-sign version can accidentally agree with multiplication for two packs of two, and why one passing example is weak evidence.

Reduce help deliberately

Completion task: Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual output, any error and what changed. If working on paper, label both results as traces.

Teacher check: Expected 24 then 30. Accept actual errors with an accurate account; do not replace observations. Record the changed pack count, saved file and run or trace status. Two matching cases do not show all inputs are handled.

Start by identifying the given inputs together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood. For partners, alternate explainer and checker; collect a response from every learner before discussion.

Individual reasoning check

Use the worked case for From a paper algorithm to Python. Proposed explanation: “An equals sign displays the answer”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: Assignment binds a name; print produces the displayed output.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class. If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

Pictorial practice question

Match each paper quantity to its Python name. Why does the spare count get added once, and what prints when spare is five?

Reasoned teacher answer

The names map trays→4, per_tray→6 and spare→2. The spare count is outside the identical trays, so total is $24+2=26$. With spare=5, print displays 29.

Changed-case reasoning: The printed total becomes 29. $4*6$ remains 24; adding five gives 29.

Collect each learner’s explanation before discussion. Use the first missing connection between input, deciding step and result to choose support. This question is worked-case practice, not fresh mastery evidence.

4. Run the investigation and diagnose misconceptions

An equals sign displays the answer: Assignment binds a name; print produces the displayed output.

A successful run means the requirement was met: Compare the output with the paper rule and expected result.

Python guesses the intended order: A name must be assigned before this script reads it.

Typing a program trains an AI model: These statements apply an explicit multiplication rule.

Use Section 9 for the evidence ledger. Add a boundary or missing-input case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

Match the support to the first error

What the response shows · Next teaching action

What the response shows: Missing or misread facts

Next teaching action: Read the case aloud, mark supplied versus unknown facts, and let the learner

restate it.

What the response shows: Wrong procedure

Next teaching action: Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.

What the response shows: Arithmetic or implementation error

Next teaching action: Trace intermediate values or lines; preserve the first mismatch and retry that step.

What the response shows: Correct result without a reason

Next teaching action: Ask for a trace and a changed case before marking independent understanding.

What the response shows: An unsupported wider claim

Next teaching action: Ask which supplied evidence supports the claim and which further observation would be needed.

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal. Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

5. Assess and explain the answers

Guided and exit questions

The third line assigns total but does not call print, so nothing is displayed. A first-line print(total) raises NameError in a fresh run because total has not been assigned. Exit: 11 is wrong for five groups of six; addition is a likely cause, and multiplication should give 30. Retain the old result and check the repaired file with both pack counts. Record whether the work was executed or traced.

W1 Connect the paper steps to Python

Write the complete four-line script for 4 packs of 6. Beside each line explain what it does. Which line displays information?

packs = 4; per_pack = 6; total = packs * per_pack; print(total), each on its own line. The first two bind quantities, the third computes 24 and the fourth displays 24. Assignment alone produces no displayed output in this script.

W2 Predict then observe

Make records for 4 packs of 6, then 5 packs of 6. Predict before running. Record actual output, any error and what changed. If working on paper, label both results as traces.

Expected 24 then 30. Accept actual errors with an accurate account; do not replace observations. Record the changed pack count, saved file and run or trace status. Two matching cases do not show all inputs are handled.

W3 Explain an order error

Move `print(total)` to the first line of a fresh script. Predict what happens and why. Put the lines back into an order that meets the requirement.

The first line tries to read `total` before it exists, causing `NameError` in a fresh run. Assign `packs` and `per_pack`, calculate `total`, then print it. Earlier values lingering in an interactive session can hide the problem; run the complete saved script afresh.

W4 Transfer the procedure independently

A fictional shelf has 3 trays of 7 tiles each. Write a four-line Python script using `trays`, `per_tray` and `tiles`. Predict its output. Then change only `trays` to 0 and predict again. Explain why adding the two quantities is wrong.

`trays = 3; per_tray = 7; tiles = trays * per_tray; print(tiles)`, on separate lines. Outputs 21 and then 0. Zero trays contain zero tiles. Addition would give 10 then 7, not the number in equal groups.

W5 Keep an honest conclusion

Explain the difference between a successful run and a correct calculation. State one limit of this program and what you would keep so another person can check your work.

A successful run means the execution finished without a reported error; correctness requires matching the stated total-count rule. The script assumes supplied whole nonnegative quantities and does not validate arbitrary input. Keep source, quantities, expected and actual output and whether it was run or traced.

Independent assessment and fresh retry

W4 must give a complete ordered script and explain outputs 21 and 0 using equal groups. Distinguish an independent paper trace from successful use of the editor; record both skills separately. After support, retry with 2 boxes of 9 pencils, then 0 boxes: 18 and 0. Ask for the print line and why it follows the calculation.

Assess independent transfer

Selected case: Predict the printed value. Change only `packs` to 5 and predict again. Explain which statements apply a written rule.

```
packs = 3
per_pack = 7
```

```
total = packs * per_pack  
  
print(total)
```

Selected case answer

21, then 35. The multiplication rule is written; changing an input is not fitting an AI model.

Dimension · Evidence required

Dimension: Explanation

Evidence required: Names the applicable idea or procedure and ties it to the supplied facts.

Dimension: Trace and result

Evidence required: Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.

Dimension: Limits

Evidence required: States the assumptions, missing evidence or conditions under which the result would change.

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace. Do not penalise an honestly recorded model failure if the learner correctly explains its evidence and limits.

Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case. Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

6. Connect the project, support and next lesson

Add a labelled from a paper algorithm to python evidence sheet to Python readiness portfolio. Show how you can translate a short paper calculation into a python script. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Offer four separate line cards, a source/output label and a multiplication sketch. Allow a paper trace when devices are unavailable, with execution practice still pending. Do not supply the independent transfer

numbers' answers. Extension: explain why a plus-sign version can accidentally agree with multiplication for two packs of two, and why one passing example is weak evidence.

Week 2 studies values, variable names and types, including the difference between numeric data and text.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

Plan the delayed return

Suggested return points in this level: python-bridge/2, python-bridge/4, python-bridge/7. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used. For a new level, revisit the previous capstone before checking the new prerequisites.

Record · Teacher evidence

Record: Learner code and date

Teacher evidence: _____

Record: Concept and case used; prior exposure

Teacher evidence: _____

Record: Explanation / trace / limits

Teacher evidence: _____

Record: Support and first repair

Teacher evidence: _____

Record: Changed case and delayed outcome

Teacher evidence: _____

Record: Next teaching action

Teacher evidence: _____

Use the evidence to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

Project evidence from this case

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.